Simulation Design

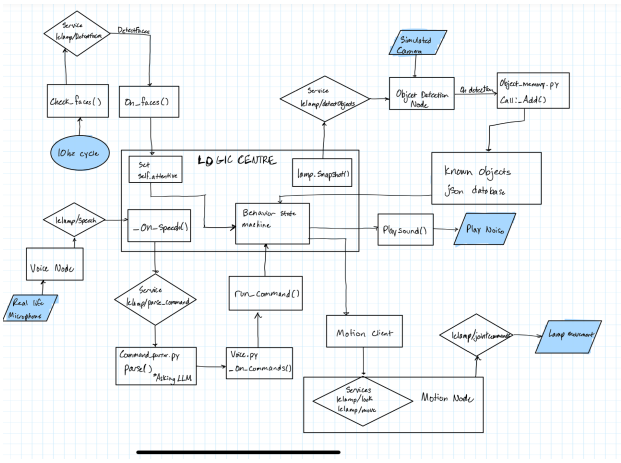


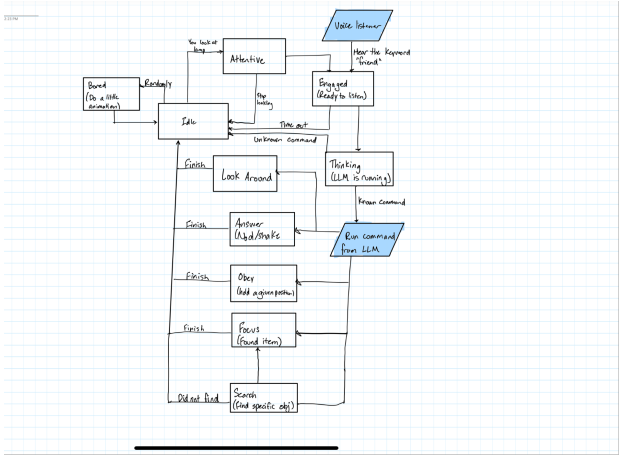
Instead of solely giving the robot the webcam to see, I decided that I wanted to place the robot inside a virtual simulation and make the webcam only one section of its vision. To do this, I added a TV inside the simulation that displays what the real life webcam sees and placed it directly in front of the robot. This lets the robot interact with me, while still having different objects in its environment to explore and interact with.

Software Architecture

I decided on running everything using ROS2 Jazzy, as it has great Ubuntu 24.04 support, lots of options for plugins, and it's something I'm very comfortable with personally. Every part of the robot runs as its own node (camera, simulation, face detection, object detection, voice, LLM, sound, and motion), so a slow LLM answer never freezes the robot's movement or vision. The perception nodes only keep the newest camera frame, so nothing builds up a backlog. Motion runs in a separate node that owns the joints at 30 Hz and refuses unsafe postures. Since the target laptop has no GPU, every model was chosen to run on the CPU. The simulation I wanted to use was MuJoCo, since it's a lightweight, an easy to use simulator, and it's also listed in your job posting. It renders on the CPU at around 30 Hz, and the whole stack installs with a single setup script.

The photo on the right shows the design of the robot's logic centre. The whole design revolves around one node named `logic_node`, defined in `logic_node/node.py`. There are two halves to this node, one side which deals with all of the services, and one which operates the behavior state machine. The service side deals with the audio parsing, facial detection, LLM querying, object detection, and memory storage through either services, or timed topics. All of this information feeds into the behavior state machine which then moves the robot, changes the light color, and plays the robot noises.





The photo on the left describes the behavior state machine implementation. All the code for this is described in `lelamp_logic/logic_node/behavior.py`. The default behavior is **Idle**, which is where it's waiting for the user to interact with it. Randomly during this, it may run a small bored animation which helps it feel a little more alive while not being interacted with. The next most central behavior is **Engaged** which is where the robot's light goes to max brightness, it stands up straight, and it begins to listen for voice commands. Engaged can be accessed by either saying the keyword "Friend", or staring at the robot long enough while it is in **Attentive** mode. Once a voice command is heard, it goes into **Thinking** mode while the LLM is coming up with a response. If a response is found, it goes into one of many commands, and then back to Idle.

Reliability

To test the reliability of the system, I ran a live demo where I measured engagement reliability, average response time to spoken questions, and RAM and CPU usage. The trial consisted of 10 activations via facial recognition, 10 activations via keyword, and 15 random questions. Here are the results:

- **Engagement reliability:** Facial detection: 10/10, wake word alone: 10/10, question response: 14/15. The one error in questions response was caused by the voice transcriber mishearing a word, causing the LLM to ignore the command.
- **Average response time:** The median response time was 2.25 seconds from end of speech to the command being run. This time consists of a silence buffer to detect the end of the sentence, the time it takes to turn the speech into text, and then the time it takes for the LLM to respond with a decision on what to do.
- **CPU and RAM usage:** the peak CPU usage was 465% of a single core, meaning it was just over the 4 core maximum. The two biggest contributors were the MuJoCo sim at 210% of one core, and the LLM at a peak value of 161% of a single core. The RAM on the other hand peaked at 4.36GB with the two biggest contributors at 2.0GB and the object detection node at 1.8GB, the total being well under the machine's limit.

Known Limitations

- **Response latency:** Every spoken command waits about 2.25 seconds, and around 1.4 seconds of that is the 1.5B LLM running on the CPU. A larger, more accurate model, unless hosted remotely, would be noticeably slower, so the lamp trades accuracy for speed.
- **Limited Voice Action Interface:** The interface between the LLM and the lamp is quite limited. It works by sending a prebuilt list of options to the LLM, which responds with a json command. The number of options the LLM has is quite limited, and each option has to be hard coded. With a stronger setup and more time I would look into training a VLA model which would allow the lamp to do much more, and seem much more natural.
- **Fragile speech and setup:** The transcriber sometimes mishears words and an unrecognized phrase drops the lamp back to idle. Without seeing the debugging logs, it would seem as if the robot just ignored your question. This could be fixed with a stronger voice transcriber, however there is no CPU headroom to spare.